

实验六第二章伪代码

本文件用于存放实验报告第二章“最大流算法基础与可行流思想”中需要插入的伪代码。算法主体使用英文描述，注释使用中文说明，便于在 Word 报告中截图插入。

算法 1 Residual Network Construction

Require: Directed graph $G = (V, E)$, capacity $c(u, v)$, current flow $f(u, v)$

Ensure: Residual graph $G_f = (V, E_f)$ with residual capacity c_f

```

1:  $E_f \leftarrow \emptyset$ 
2: for each edge  $(u, v) \in E$  do
3:    $forward \leftarrow c(u, v) - f(u, v)$ 
4:   if  $forward > 0$  then
5:     add edge  $(u, v)$  to  $E_f$  with capacity  $forward$            ▷ 正向边表示还能继续增加的流量
6:   end if
7:   if  $f(u, v) > 0$  then
8:     add edge  $(v, u)$  to  $E_f$  with capacity  $f(u, v)$            ▷ 反向边表示可以撤回并重新分配的流量
9:   end if
10: end for
11: return  $G_f$ 

```

算法 2 Edmonds-Karp Maximum Flow

Require: Flow network $G = (V, E)$, source s , sink t , capacity $c(u, v)$

Ensure: Maximum flow value $maxFlow$

```

1: for each edge  $(u, v) \in E$  do
2:    $f(u, v) \leftarrow 0$                                            ▷ 初始时所有边流量为 0
3: end for
4:  $maxFlow \leftarrow 0$ 
5: while there exists an augmenting path  $P$  from  $s$  to  $t$  found by BFS do
6:    $\Delta \leftarrow \min\{c_f(u, v) \mid (u, v) \in P\}$              ▷ 路径瓶颈容量
7:   for each edge  $(u, v) \in P$  do
8:     if  $(u, v)$  is a forward edge then
9:        $f(u, v) \leftarrow f(u, v) + \Delta$                        ▷ 沿正向边增加流量
10:    else
11:       $f(v, u) \leftarrow f(v, u) - \Delta$                        ▷ 沿反向边撤回部分流量
12:    end if
13:  end for
14:   $maxFlow \leftarrow maxFlow + \Delta$ 
15: end while
16: return  $maxFlow$ 

```

算法 3 Dinic Maximum Flow

Require: Flow network $G = (V, E)$, source s , sink t , capacity $c(u, v)$
Ensure: Maximum flow value $maxFlow$

```

1: initialize all flows to 0
2:  $maxFlow \leftarrow 0$ 
3: while BFS can build a level graph from  $s$  to  $t$  do
4:    $level[v] \leftarrow$  distance from  $s$  to  $v$  in residual graph  $\triangleright$  构造分层图，记录每层节点到源点的最短距离
5:   reset current arc pointer  $cur[v]$  for every vertex  $v$   $\triangleright$  当前弧优化，避免重复扫描无效边
6:   while  $pushed \leftarrow \text{DFS}(s, \infty, level, cur)$  is greater than 0 do
7:      $maxFlow \leftarrow maxFlow + pushed$   $\triangleright$  在同一个分层图中不断寻找阻塞流
8:   end while
9: end while
10: return  $maxFlow$ 

```

算法 4 DFS in Dinic Level Graph

Require: Current vertex u , available flow $flow$, level array $level$, current arc array cur
Ensure: Flow successfully pushed from u to t

```

1: if  $u = t$  then
2:   return  $flow$ 
3: end if
4: for each edge  $(u, v)$  starting from  $cur[u]$  do
5:   if  $c_f(u, v) > 0$  and  $level[v] = level[u] + 1$  then
6:      $pushed \leftarrow \text{DFS}(v, \min(flow, c_f(u, v)), level, cur)$   $\triangleright$  沿分层图递归推流
7:     if  $pushed > 0$  then
8:        $c_f(u, v) \leftarrow c_f(u, v) - pushed$ 
9:        $c_f(v, u) \leftarrow c_f(v, u) + pushed$   $\triangleright$  更新正向边与反向边残量容量
10:      return  $pushed$ 
11:     end if
12:   end if
13:    $cur[u] \leftarrow cur[u] + 1$   $\triangleright$  当前弧优化，跳过已无法推流的边
14: end for
15: return 0

```

算法 5 Build Baseball Flow Network

Require: Teams $0, \dots, n-1$, wins w , remaining games r , game matrix g , target team x

Ensure: Flow network N_x , total game capacity G_x , team-node map $teamNode$

```

1:  $M_x \leftarrow w_x + r_x$  ▷ 待判断球队的最高可能胜场
2: for each team  $i \neq x$  do
3:   if  $w_i > M_x$  then
4:     return trivial elimination by team  $i$  ▷ 平凡淘汰，直接返回
5:   end if
6: end for
7: create source  $s$  and sink  $t$ 
8:  $G_x \leftarrow 0$ 
9: for each team  $i \neq x$  do
10:   create team node  $T_i$ 
11:    $teamNode[i] \leftarrow T_i$ 
12:   add edge  $(T_i, t)$  with capacity  $M_x - w_i$  ▷ 限制球队 i 最多还能赢的场数
13: end for
14: for each pair of teams  $(i, j)$  with  $i < j, i \neq x, j \neq x$  do
15:   if  $g_{ij} = 0$  then
16:     continue ▷ 没有剩余比赛，不需要建立比赛节点
17:   end if
18:   create game node  $G_{ij}$ 
19:   add edge  $(s, G_{ij})$  with capacity  $g_{ij}$  ▷ 这组剩余比赛必须产生的胜场数
20:   add edge  $(G_{ij}, T_i)$  with capacity  $INF$ 
21:   add edge  $(G_{ij}, T_j)$  with capacity  $INF$  ▷ 胜场只能分配给参赛双方
22:    $G_x \leftarrow G_x + g_{ij}$ 
23: end for
24: return  $N_x, G_x, teamNode$ 

```
